

Project Proposal

Extending a Framework for Neural Algorithmic Reasoning with Multiple Solutions

Introduction

Neural Algorithmic Reasoning (NAR) is a paradigm that trains neural networks (NNs) to reproduce the execution of classical algorithms. Canonical NAR models, typically based on the encode-process-decode (EPD) architecture, usually learn a single trajectory or output. However, many algorithms included in benchmarks like the CLRS Algorithmic Reasoning Benchmark, such as Depth-First Search (DFS) and Bellman-Ford (BF), naturally admit multiple correct solutions for a given input graph, often arising due to arbitrary tie-breaking rules.

The paper, Neural Algorithmic Reasoning with Multiple Correct Solutions (Kujawa et al., 2025), presents a methodology to address this limitation by training an NN to predict the underlying distribution of possible solutions. This is achieved by generating diverse training data through randomized tie-breaking during algorithm execution and then employing stochastic sampling to extract multiple unique and valid solutions from the NN’s output distribution.

This project is a software engineering implementation focused on extending this multi-solution NAR framework. The core objective is to integrate the Breadth-First Search (BFS) algorithm into this system, leveraging the successful implementation methodologies already established for BF and DFS. BFS is a natural candidate as it is a fundamental graph traversal algorithm included in the CLRS benchmark and, like DFS, frequently yields multiple correct spanning trees on unweighted graphs.

The work will focus on three major components:

1. Implementing the data generation pipeline for BFS, incorporating randomized tie-breaking, following the format used in related BF/DFS research.
2. Training a standard NAR model (e.g., a Message-Passing Neural Network (MPNN) processor) to predict the BFS solution distribution.

3. Implementing and evaluating stochastic sampling techniques to reliably extract multiple unique and valid BFS solutions.

Starting point

This project will build upon the established CLRS framework from the paper The CLRS Algorithmic Reasoning Benchmark (Veličković et al., 2022), as well as the algorithms described in the paper Neural Algorithmic Reasoning with Multiple Correct Solutions (Kujawa et al., 2025).

I am familiar with fundamental algorithms, including graph traversal and shortest path algorithms (e.g., Bellman-Ford), often used in NAR tasks. However I do not have any prior experience working with Graph Neural Networks, nor any experience working with Deep Learning frameworks like PyTorch or JAX.

I have no prior experience implementing the specific randomized data generators or the multi-solution extraction framework described in the target paper. **No code specific to this project, including GNN architectures or data generators, will be written prior to the start of the project in Michaelmas.**

Core project

The project is structured around adapting the established NAR methodology for BFS

- **Data Generation and Solution Space:** I will implement an expert policy for BFS that, when computing the predecessor array (π), uses **randomized tie-breaking** when selecting which node to explore next among eligible candidates. These multiple resulting predecessor arrays are aggregated to produce the supervised signal: a **Parent Probability Distribution matrix** ($\tilde{P}$). The generation process should target graph sizes consistent with the CLRS-30 benchmark ($N \leq 64$).
- **NAR Model Implementation:** I will train a standard Graph Neural Network (GNN) model using the EPD framework, in which the processor component is a Message-Passing Neural Network (MPNN). The model will be trained using supervised learning to predict the probability distribution $\hat{P}$ over parent pointers, minimizing cross-entropy or similar loss against the ground-truth distribution $\tilde{P}$.
- **Extracting Multiple Solutions:** Extracting candidate predecessor arrays from the predicted distribution $\hat{P}$ using algorithm-specific stochastic sampling methods, in this case for BFS.
- **Evaluation of Solutions:** Rigorously evaluating the diversity and correctness of the extracted solutions.

Success criteria

The project will be considered a success if the following goals are demonstrably achieved:

1. **BFS Data Pipeline:** A data generation utility for BFS, incorporating randomized tie-breaking, is successfully implemented and shown to produce valid Parent Probability Distribution matrices ($\tilde{P}$) for a representative set of small and medium graphs.
2. **NAR Model Training:** The MPNN model is successfully trained on the BFS distribution data.
3. **Multiple Solution Sampling:** Implement methods for sampling valid BFS solutions from a parent distribution.
4. **Evaluating Solution Validity:** Implement a method to evaluate the correctness of sampled BFS solutions, ensuring they form valid BFS graph traversals.

Possible extensions

- **Implementing Other Algorithms:** Implement a similar training and evaluation pipeline for one or more other algorithms from the CLRS benchmark that also admit multiple solutions, such as Prim’s algorithm for Minimum Spanning Tree (MST) or Task Scheduling.

This would allow us to further investigate the generality of this multi-solution NAR framework. In particular this would provide more insight into the extent to which (i) certain algorithms (such as DFS, as mentioned in the paper) are hard to emulate with NAR in the first place, and whether (ii) encoding multiple solutions as a probability distribution can correctly optimise for correct solutions.

- **Multi-Task Training:** Investigate how training the model on multiple algorithms (with multiple solutions) simultaneously, allowing the network to share representations across different tasks, could affect its performance.
- **Alternative NAR Architecture:** Implement the NAR model using a Pointer Graph Network (PGN) processor, as PGNs have shown competitive generalisation out-of-distribution compared to MPNNs on CLRS tasks.

Work plan

Michaelmas Term

- **Weeks 1-2 (9 Oct - 22 Oct 2025)**
 - Set up GitHub repository.
 - Read up on and understand the codebase of the CLRS-30 benchmark.
 - Familiarise with the existing BF/DFS multi-solution framework and MPNN implementation.
 - Configure machine learning environment (PyTorch, PyTorch Geometric, etc.).
 - Conduct literature review of related NAR work.
 - **Milestone:** Repository and environment set up; firm understanding of the BF/DFS system.

- **Weeks 3-4 (23 Oct - 5 Nov 2025)**
 - Implement the BFS data generation pipeline with randomized tie-breaking.
 - Verify correctness and variability of generated predecessor arrays (π).
 - Aggregate multiple runs to produce the Parent Probability Distribution ($\tilde{P}$).
 - **Milestone:** BFS data generation component implemented and validated.
- **Weeks 5-6 (6 Nov - 19 Nov 2025)**
 - Train the MPNN-based NAR model on the BFS dataset and conduct initial evaluations.
 - **Milestone:** MPNN model implemented and initial training completed.
- **Weeks 7-8 (20 Nov - 3 Dec 2025)**
 - Implement initial method for sampling multiple BFS solutions from the predicted distribution.
 - Implement algorithm for solution validation.
 - Test the system's ability to extract multiple unique BFS solutions.
 - **Milestone:** Implement one method for sampling solutions and validating correctness.

Christmas Vacation

- **Weeks 1-2 (4 Dec - 17 Dec 2025)**
 - 2-week break (no work planned).
- **Weeks 3-4 (18 Dec - 31 Dec 2025)**
 - Implement additional methods for sampling solutions.
 - Evaluate performance of different sampling methods.
 - **Milestone:** Core BFS framework implemented.
- **Weeks 5-6 (1 Jan - 14 Jan 2026)**
 - Finish any incomplete implementation of the core project.
 - **Milestone:** Core project completed.
- **Week 7 (18 Dec - 31 Dec 2025)**
 - Begin progress report.
 - Choose extensions to work on.
 - **Milestone:** Finalize set of chosen extensions.

Lent Term

- **Weeks 1-2 (22 Jan - 4 Feb 2026)**
 - Finalise progress report.

- **Milestone:** Submit progress report by 6 Feb 2026.
- **Weeks 3-4 (5 Feb - 18 Feb 2026)**
 - Implement Extension I
 - **Milestone:** Implementation and evaluation for Extension I finished.
- **Weeks 5-6 (19 Feb - 4 Mar 2026)**
 - Implement Extension II
 - **Milestone:** Extension II implementation and evaluation complete.
- **Weeks 7-8 (5 Mar - 18 Mar 2026)**
 - Write the Introduction and Preparation chapters of the dissertation.
 - Finalise unit testing, documentation and comments.
 - **Milestone:** Complete Introduction and Preparation chapters of dissertation.

Easter Vacation

- **Weeks 1-2 (19 Mar - 1 Apr 2026)**
 - Write the Implementation section of the dissertation.
 - **Milestone:** Complete Implementation section.
- **Weeks 3-4 (2 Apr - 15 Apr 2026)**
 - Write the Evaluation and Conclusion sections of the dissertation.
 - **Milestone:** Complete Evaluation and Conclusion sections.
- **Weeks 5-6 (16 Apr - 29 Apr 2026)**
 - Send dissertation for feedback; edit as required.
 - Buffer time for any previous incomplete work.
 - **Milestone:** Complete initial draft of full dissertation.

Easter Term

- **Weeks 1-2 (30 Apr - 13 May 2026)**
 - Final edits and proofreading of the dissertation.
 - **Milestone:** Submission of dissertation.

Resource declaration

My primary working device will be a MacBook Pro (2021, Apple M1, 16GB RAM, macOS Sequoia). For writing, I will use a combination of Overleaf and Google Docs, with regular backups being made to my University OneDrive account. Code will be written in Python using an IDE such as VSCode, with regular GitHub version control. While my laptop can run smaller tests and is suitable for smaller datasets, it may not handle more demanding

tasks (e.g., multi-task training). For those, I will use Google Colab or request GPU access from the Bioinformatics Group at the Computer Laboratory.